RescueSync – API del Sistema Nacional

Manual de Pruebas – Colección Postman

Versión 1.1

1. Introducción

Este documento describe cómo importar, configurar y ejecutar la colección Postman **RescueSync – API del Sistema Nacional**. La colección contiene 38 casos de prueba distribuidos en 7 carpetas que cubren autenticación JWT, control de acceso por roles, catálogo de recursos, padrón de ONGs, gestión de inventario, compromisos y liberación de recursos, control de concurrencia y evaluación de ofertas.

Los tests están escritos en JavaScript (Postman Sandbox) y se ejecutan automáticamente al correr la colección en modo *Collection Runner*. Cada request valida el código HTTP de respuesta, la estructura del cuerpo JSON y, en varios casos, guarda valores en **variables de colección** que son consumidos por requests posteriores.

2. Archivos incluidos

Archivo	Descripción
sistema-nacional.postman_collection.json	Colección con los 38 requests y sus tests.
local.postman_environment.json	Entorno para desarrollo local (baseUrl → http://localhost:8000).
render.postman_environment.json	Entorno para producción en Render. Requiere completar contrasenaAdmin y clientSe

3. Prerrequisitos e importación

3.1 Software necesario

Postman Desktop ≥ 10 (o Postman Web). No se requiere ninguna herramienta adicional: todos los scripts de generación de datos y validación están embebidos en la colección.

3.2 Pasos de importación

1. Abrir Postman y seleccionar **Import** (botón superior izquierdo).
2. Arrastrar o seleccionar **sistema-nacional.postman_collection.json**.
3. Repetir para el entorno deseado: **local.postman_environment.json** o **render.postman_environment.json**.
4. En el selector de entorno (esquina superior derecha), activar el entorno importado.

3.3 Completar credenciales del entorno

Antes de ejecutar la colección, completar los siguientes valores en la pestaña *Current Value* del entorno seleccionado:

Variable	Valor esperado	Dónde obtenerlo
usuarioAdmin	admin-sn	Fijo. No cambia entre entornos.
contrasenaAdmin	(contraseña del admin)	Variable SEMILLA_CONTRASENA_ADMIN del .env de producción (Render → Environment).
clientIdBonita	bonita-rescuesync	Fijo. No cambia entre entornos.

clientSecretBonita	(secreto de Bonita)	Variable SEMILLA_SECRETO_BONITA del .env de producción (Render → Environment).

■ **Nota:** las variables *tokenAdmin*, *tokenBonita*, *cuitNueva*, *itemId*, *certificacionId* y *emergenciaPostmanId* se populan **automáticamente** durante la ejecución. No es necesario completarlas.

4. Variables dinámicas y cadena de dependencias

La colección utiliza variables de colección para encadenar resultados entre requests. Entender esta cadena es clave para ejecutar las carpetas en el orden correcto y para interpretar errores cuando se corre una carpeta de forma aislada.

Variable	Generada en	Usada en	Cómo se genera
tokenAdmin	1. Autenticación → Login administrador (JSON)	Todos los requests que requieren autenticación	Script test: extrae access_token de la respuesta y lo guarda
tokenBonita	1. Autenticación → Login Bonita (OAuth2)	Requests que simulan al core de Bonita	Script test: extrae access_token de la respuesta OAuth2.
tokenAdulterado	2. Protección → Token adulterado	Mismo request (se usa de inmediato)	Pre-request script: invierte los bytes de la firma del JWT de
itemId	3. Catálogos → Disponibilidad global de agua	4. Gestión → Cargar ítem a inventario	Script test: toma el id del primer elemento devuelto por /recu
certificacionId	3. Catálogos → Catálogo de certificaciones	4. Gestión → Cargar certificación y ver detalle	Script test: toma el id de la primera certificación de /certifica
cuitNueva	5. Padrón → Alta de ONG (onboarding)	4. Gestión, 5. Padrón (habilitación histórica)	Pre-request: genera CUIT aleatorio (30 + 9 dígitos). Garanti
emergenciaPostmanId	6. Compromiso → Compromiso	6. Compromiso → Liberación	Pre-request: 'emergencia-postman-' + Date.now(). Cada run

■ **Importante:** si se corre la carpeta **4 (Gestión de organizaciones)** de forma aislada sin haber ejecutado antes la carpeta 3 (Catálogos) y la carpeta 5 (Padrón → Alta de ONG), las variables *itemId*, *certificacionId* y *cuitNueva* estarán vacías y los requests fallarán con 404 o 422. Siempre correr la colección completa en orden o respetar el orden indicado en la sección 5.

5. Orden de ejecución y modo de uso

El *Collection Runner* de Postman respeta el orden en que aparecen las carpetas y requests dentro de la colección. Se recomienda ejecutar **toda la colección de una vez** sin modificar el orden.

Paso	Carpeta (en orden de ejecución)	Qué hace
1	1. Autenticación	Login JSON y OAuth2. Guarda tokenAdmin y tokenBonita.
2	2. Protección de endpoints	Verifica que sin token o con token adulterado se rechaza (401/403).
3	3. Consultas de catálogo	GETs de certificaciones, ONGs y disponibilidad. Guarda itemId y certificacionId.

4	5. Padrón de ONGs	Alta de ONG con CUIT aleatorio, habilitación, historial. Guarda cuitNueva. También prueba los
5	4. Gestión de organizaciones	Carga ítem y certificación a la ONG creada en paso 4. Requiere itemId, certificacionId y cuitNu
6	6. Compromiso/Liberación	Compromiso de agua sobre Médicos Sin Fronteras (CUIT semilla). Dry-run (validar). Idempoter
7	7. Evaluación de ofertas	Evaluación de oferta de Médicos Sin Fronteras. Solo lectura, sin efectos secundarios.

Cómo lanzar el Collection Runner:

1. Click derecho sobre la colección → **Run collection**.
2. Verificar que el entorno correcto esté seleccionado en el desplegable *Environment*.
3. Dejar la opción *Run order* en el orden por defecto (no reordenar).
4. Click en **Run RescueSync – API del Sistema Nacional**.
5. Al finalizar, el runner muestra el resultado de cada test. El resultado esperado es **38/38 passed**.

6. Descripción detallada por carpeta

6.1 Autenticación (6 requests, 6 tests)

Prueba el endpoint **POST /login** en sus dos modos y el endpoint **GET /cuentas/yo**. Debe ejecutarse primero porque guarda los tokens que usan todas las carpetas siguientes.

Request	Método	Resultado esperado
Login administrador (JSON)	POST /login	200 – token Bearer con Cache-Control: no-store.
Login Bonita (OAuth2 client creds)	POST /login	200 – token Bearer OAuth2 (grant_type=client_credentials).
Login con contraseña incorrecta	POST /login	401 NO_AUTENTICADO.
Login OAuth2 con secreto incorrecto	POST /login	401 invalid_client (RFC 6749).
Cuenta del token (administrador)	GET /cuentas/yo	200 – rol: administrador.
Cuenta del token (Bonita)	GET /cuentas/yo	200 – rol: sistema, con campo client_id.

6.2 Protección de endpoints – RBAC (4 requests, 4 tests)

Verifica que los endpoints estén protegidos contra acceso sin token, con token adulterado y con un rol sin permisos suficientes. El *pre-request script* del request "Token adulterado" invierte los bytes de la firma del JWT del administrador para simular una falsificación.

Request	Escenario	Resultado esperado
Sin token	GET sin cabecera Authorization	401 NO_AUTENTICADO.
Token adulterado	GET con JWT cuya firma fue invertida	401 NO_AUTENTICADO.
Bonita intenta dar de alta ONG	POST /ongs con token de rol sistema	403 PERMISO_DENEGADO.
Bonita intenta suspender ONG	PATCH /ongs/.../habilitacion (rol sistema)	403 PERMISO_DENEGADO.

6.3 Consultas de catálogo (4 requests, 7 tests)

Consultas de solo lectura que validan la estructura paginada de las respuestas. El test de "Disponibilidad global de agua" guarda el *id* del primer ítem del catálogo en la variable **itemId**, que se usa en la carpeta 4 (Gestión). El test de "Catálogo de certificaciones" guarda **certificacionId** de la misma forma.

Request	Endpoint	Qué valida
Catálogo de certificaciones	GET /certificaciones	200 + estructura paginada. Guarda certificacionId.
ONGs habilitadas	GET /ongs	200 + lista de organizaciones con id, cuit, nombre.
Disponibilidad global de agua	GET /recursos/disponibilidad?tipo=agua	200 + cada ítem tiene consumible, total, comprometido, disponible ≥ 0
Disponibilidad de una ONG	GET /ongs/{cuit}/recursos/disponibilidad	200 + estructura paginada de inventario de Médicos Sin Fronteras.

6.4 Padrón de ONGs (10 requests, 10 tests) – carpeta «5» en la colección

Prueba el ciclo completo de alta y gestión de habilitación. El CUIT de la ONG nueva se genera aleatoriamente en el *pre-request script* del request "Alta de ONG": `30 + 9 dígitos aleatorios`. Esto garantiza que cada run cree una organización distinta y no colisione con los datos semilla.

Request	Resultado esperado
Consultar ONG suspendida (Bonita)	200 – estado_habilitacion: suspendida, habilitada: false. (Cruz Roja, estado semilla.)
Alta de ONG (onboarding)	201 – estado_habilitacion: pendiente. Guarda cuitNueva.
Alta duplicada (mismo CUIT)	409 CONFLICTO_DE_ESTADO – campo: cuit.
Alta con CUIT inválido	400 SOLICITUD_INVALIDA.
Habilitar la ONG nueva	200 – estado_habilitacion: habilitada.
Habilitarla otra vez	409 CONFLICTO_DE_ESTADO (ya estaba habilitada).
Suspender sin motivo	400 SOLICITUD_INVALIDA (motivo requerido al suspender).
Rehabilitar Cruz Roja	200 – habilitada: true. (Necesario para la siguiente.)
Volver a suspender Cruz Roja	200 – estado_habilitacion: suspendida. (Restaura estado semilla.)
Historial de habilitación de Cruz Roja	200 – más de 2 registros, último: suspendida.

■ Cruz Roja (*30-99999999-4*) queda **suspendida al final de esta carpeta**. Ese es su estado en los datos semilla. No es un error: la colección rehabilita y vuelve a suspender deliberadamente para dejar la base en el mismo estado que al inicio.

6.5 Gestión de organizaciones (3 requests, 3 tests) – carpeta «4» en la colección

Carga un ítem de inventario y una certificación sobre la ONG creada en la carpeta anterior. **Requiere que itemId, certificacionId y cuitNueva estén definidas**; se deben haber ejecutado antes las carpetas 3 y 5 (Catálogos y Padrón).

Request	Resultado esperado
Cargar ítem al inventario	201 – item_id correcto, cantidad_disponible: 100.
Cargar certificación y nivel	201 – certificacion_id correcto, nivel_certificacion: alta.
Nivel de certificación inválido	400 – nivel 'ALTO' (mayúsculas) no es un valor aceptado.

6.6 Compromiso / Liberación / Concurrencia (7 requests, 9 tests)

Carpeta central del sistema. Prueba el ciclo completo de compromiso y liberación de recursos sobre **Médicos Sin Fronteras** (CUIT semilla *30-65432198-7*). El ID de emergencia se genera dinámicamente: `'emergencia-postman-' + Date.now()`, lo que garantiza que cada run use un ID distinto y no haya colisión con ejecuciones anteriores.

Request	Qué prueba
Compromiso	Crea el compromiso. Responde 200 (idempotente) o 201 (nuevo). Verifica que al menos un ítem te
Validar restricciones – OK	Dry-run (/recursos/compromisos/validar). No modifica datos. Responde 200 con valido: true y moti
Validar restricciones – Stock insuficiente	Dry-run con paramédicos (RSR-PARAMEDICOS) en cantidad imposible. Responde 200 con valido
Validar restricciones – Consumible cantidad extrema	Dry-run con agua en cantidad extrema (consumible). Responde 200 con valido: true: los consumib
Reintento de compromiso (Idempotencia)	Envía exactamente el mismo body que el primer Compromiso. Responde 200 con repetido: true. N
Liberación	POST /recursos/compromisos/{emergenciaId}/ongs/30-65432198-7/finalizacion. Responde 200 co
Concurrencia	GET de disponibilidad de Scouts Pergamino (CUIT semilla) seguido de dos POST paralelos ejecut

■ **El endpoint /validar es un dry-run puro:** no crea ni modifica ningún registro en la base de datos. Se puede ejecutar cualquier número de veces sin efecto secundario. Los recursos consumibles (ej. agua, medicamentos) no tienen límite de stock: la validación siempre retorna valido: true para ellos independientemente de la cantidad solicitada.

6.7 Evaluación de ofertas (4 requests, 4 tests)

Valida la lógica de evaluación de oferta de Médicos Sin Fronteras. Todos los requests son de solo lectura (POST /ofertas/evaluacion no persiste datos).

Request	Escenario	Resultado esperado
Caso principal o apoyo	ONG habilitada con recursos suficientes	200 – rol: principal o apoyo.
ONG no registrada	CUIT inexistente en el padrón	200 – rechazado: organización no registrada.
Rechazo por cantidad insuficiente	Recursos disponibles por debajo del mínimo	200 – rechazado: cantidad_insuficiente.
Rechazo por falta de certificación	ONG sin certificación requerida	200 – rechazado: certificacion_insuficiente.

7. Datos semilla cargados en la base de datos

La base de datos (Supabase – PostgreSQL) cuenta con un conjunto de datos iniciales cargados automáticamente al ejecutar las migraciones. Estos datos son los que usan los tests de la colección. A continuación se listan las entidades disponibles.

7.1 Organizaciones (ONGs)

CUIT	Nombre	Estado	Rol en los tests
30-99999999-4	Cruz Roja Argentina	suspendida	Pruebas de habilitación/historial. Se rehabilita y vuelve a suspend
30-65432198-7	Médicos Sin Fronteras	habilitada	Compromiso y liberación de recursos, evaluación de oferta.
30-71234567-8	Techo Argentina	habilitada	Disponible como organización habilitada en listados.
30-88765432-1	Banco de Alimentos CABA	habilitada	Disponible como organización habilitada en listados.
30-99999999-6	Scouts Pergamino	habilitada	Prueba de concurrencia (control de locking pesimista).

7.2 Catálogo de recursos e inventarios

Los ítems del catálogo tienen un código de recurso (*RSR-**) y un tipo (agua, medicamentos, personal, etc.). Los consumibles (agua, medicamentos) no tienen límite de stock: pueden comprometerse en cualquier cantidad. Los no consumibles (personal, vehículos) sí tienen stock máximo.

Código	Nombre	Consumible	Asignado a
RSR-AGUA	Agua potable (litros)	Sí	Médicos Sin Fronteras, Scouts Pergamino
RSR-MEDICAMENTOS	Medicamentos básicos	Sí	Médicos Sin Fronteras
RSR-PARAMEDICOS	Paramédicos	No	Médicos Sin Fronteras (stock limitado)
RSR-VOLUNTARIOS	Voluntarios generales	No	Scouts Pergamino (stock limitado)
RSR-ALIMENTOS	Alimentos no perecederos	Sí	Banco de Alimentos CABA

7.3 Certificaciones del catálogo

Las certificaciones son requisitos que las ONGs deben acreditar para participar en ciertos proyectos. Tienen niveles: *baja*, *media*, *alta*.

Certificación	Niveles disponibles	ONG con certificación
Primeros auxilios	baja, media, alta	Médicos Sin Fronteras (alta)
Manejo de sustancias	baja, media	Médicos Sin Fronteras (media)
Gestión de voluntarios	baja, media, alta	Scouts Pergamino (media)

8. Idempotencia y estado de la base al finalizar el run

La colección está diseñada para dejar la base de datos en un estado **equivalente al inicial** después de un run completo. Esto permite ejecutarla múltiples veces sin necesidad de restaurar los datos.

Acción que modifica datos	Cómo se revierte o neutraliza
Cruz Roja se rehabilita (carpeta 5)	El request siguiente la vuelve a suspender. Estado final = suspendida (igual que el semilla).
Se crea una ONG nueva (carpeta 5)	CUIT aleatorio único. No interfiere con los datos semilla. Se puede ignorar o limpiar.
Se carga ítem/certificación a ONG nueva (carpeta 4)	Asociada a la ONG nueva (descartable). No afecta a las ONGs semilla.
Se comprometen recursos de Médicos Sin Fronteras (carpeta 6)	El request de liberación cierra el proyecto (proyecto_cerrado: true). Los recursos quedan lib
Los requests de Concurrencia comprometen recursos (carpeta 8)	Los dos POST se ejecutan sobre el mismo emergenciaPostmanId ya finalizado; el locking

9. Consideraciones y resolución de problemas frecuentes

• **Cold start en Render:** la instancia gratuita de Render se apaga tras 15 minutos de inactividad. El primer request puede tardar 30–60 segundos. Si ese primer request falla con timeout, correr nuevamente la colección: los tokens ya serán válidos en el segundo intento.

• **Expiración del token JWT:** el token tiene una vida útil configurada en el servidor. Si el run demora más de ese límite (raro en condiciones normales), los requests de carpetas tardías pueden recibir 401. Solución: volver a correr la colección completa.

• **Carpetas corridas en aislamiento:** si se ejecuta una carpeta sola y falla con 404 o 422, verificar que las variables dinámicas (tokenAdmin, itemId, cuitNueva, etc.) estén definidas. Ver sección 4 para la cadena de dependencias.

• **El test de Concurrencia no es un test de carga:** el script JavaScript del request "Concurrencia" ejecuta dos POST de compromiso en paralelo usando pm.sendRequest. El resultado esperado es que exactamente uno prospere y el otro sea rechazado o devuelva repetido: true, gracias al mecanismo de locking pesimista (SELECT FOR UPDATE) de la API.

• **La carpeta 4 aparece después de la carpeta 5 en la colección:** el orden de numeración en los nombres de las carpetas no coincide con el orden de aparición en el archivo JSON. El Collection Runner respeta el orden físico, que es el correcto para satisfacer las dependencias entre variables.